

BRAIN TUMOR DETECTION USING DEEP LEARNING

BY

ISAH OKHAI JEFFERY

PSC1709420

DEPARTMENT OF COMPUTER SCIENCE,

FACULTY OF PHYSICAL SCIENCES,

UNIVERSITY OF BENIN,

BENIN CITY,

EDO STATE, NIGERIA.

DECEMBER 2022

BRAIN TUMOR DETECTION WITH DEEP LEARNING

BY

ISAH OKHAI JEFFERY

PSC1709420

**A PROJECT REPORT SUBMITTED TO THE DEPARTMENT OF COMPUTER
SCIENCE, FACULTY OF PHYSICAL SCIENCES, UNIVERSITY OF BENIN, BENIN
CITY**

**IN PARTIAL FULFILMENT OF THE REQUIREMENT FOR THE AWARD OF A
BACHELOR OF SCIENCE (B.Sc.) DEGREE IN COMPUTER SCIENCE**

DECEMBER 2022

CERTIFICATION

This is to certify that this project work was carried out by **ISAH OKHAI JEFFERY** with Matriculation Number **PSC1709420** under my supervision and it is adequate and satisfactory, both in scope and content, for the award of Bachelor of Science (B.Sc) Degree in Computer Science of the University of Benin

F.O. Oliha, PhD

Project Supervisor

DATE

APPROVAL

This project work is hereby approved in partial fulfillment of the requirements for the award of Bachelor of Science (B.Sc.) Degree in Computer Science of the University of Benin

PROF. A.O. EGWALI

Head of Department

DATE

DEDICATION

I dedicate this work to God, for giving me the strength and guidance to properly carry out and complete the work and also for his protection throughout my time in the University of Benin

This work is also dedicated to my parents, for making this journey as possibly easy as they could, for encouraging me and guiding me.

This work is dedicated to my project supervisor. This work wouldn't be complete without you; you gave me proper direction and guidance.

This work is also dedicated to my set mates (INT 21H) for the camaraderie and friendship I received throughout our stay in this university it will not be forgotten.

ACKNOWLEDGEMENT

My utmost appreciation goes to God, for seeing me through all my years in school.

To my project supervisor, F.O. Oliha, PhD. for your incredible patience, motivation, support and guidance not only during the session but also during the strike, I am thankful and immensely grateful. Special gratitude goes to the Head of Department of Computer Science, Prof. (Mrs) A.O. Egwali , my project coordinator Dr. R.A. Usiobaifo ,and all the other lecturers in the Department of Computer Science, University of Benin, for their support and inspiration in one way or the other: Prof. (Mrs.) Akwukwuma, Prof. (Mrs) F.A. Egbokhare, Prof. A.A. Imiavan, Prof K.C. Ukaoha, Prof. (Mrs.) S. Konyeha, Prof. (Mrs) S.C. Chiemeke, Prof. G.O. Eguobase, Prof F.I. Amadin , Dr. F.O. Chete, Dr(Mrs).G. Aziken, Dr. E. Nwelih, Dr. E.P. Ebietomere, Dr. (Mrs) R.O. Osaseri, Mr. E.E. Obasohan, Mr. P.E.B. Imiefoh, Mr. S.O.P. Oliomogbe, Mrs. R.A. Usiobaifo, Mr. E.C. Igodan, Mr. K.O. Otkiti, Mr. F. Osagie, Miss O.L. Usiosefe, Mr. E. Obayagbona, Mrs. T. Agenmonmen, Mr. Odetayo. God bless you all. Immeasurable thanks to my parents, for their constant unwavering support, care and guidance during the course of this project work and my stay in school. Also, thanks to my siblings and friends who have offered their support and listening ears in my times of weakness. May God, in his infinite mercy, bless you all.

Table of Contents

CERTIFICATION.....	iii
APPROVAL.....	iv
DEDICATION.....	v
ACKNOWLEDGEMENT.....	vi
ABSTRACT.....	ix
CHAPTER ONE.....	1
INTRODUCTION.....	1
1.1 Background Information.....	1
1.2 Motivation.....	3
1.3 Problem Definition.....	3
1.4 Goal and Objectives.....	4
1.5 Scope of Research.....	4
1.4 Research Methodology.....	4
1.5 Research Significance.....	5
CHAPTER TWO.....	6
LITERATURE REVIEW.....	6
2.1 Brain tumors: An Overview.....	6
2.1.1 Classification Brain tumors.....	6
A. Benign tumors.....	6
B. Malignant Tumors:.....	7
2.2 Related Work.....	8
CHAPTER THREE.....	13
METHODOLOGY.....	13
3.1 Data set.....	13
3.2 DEEP LEARNING.....	14
3.2.1 Neuron.....	14
3.2.2 Artificial Neural Network.....	15
3.3 Types of Neural Networks.....	16
3.4 Deep learning models.....	17

3.4.5	Convolutional Neural Network (CNN).....	20
(A)	Why CNN is different than simple Neural Network.....	21
(B)	Layer's Used to a Build CNN Model:.....	25
(C)	Limitations of CNN:.....	35
CHAPTER FOUR.....		37
SYSTEM DEVELOPMENT.....		37
4.1 Software Implementation Tools.....		37
a.	Implementation Languages.....	37
b.	Packages.....	37
4.2 Training the Model.....		39
4.2.1	Acquiring the Dataset.....	40
4.2.2	Convolutional Layer.....	42
4.2.3	Max Pooling Layer.....	43
4.2.4	Flatten Layer.....	44
4.2.4	Fully Connected Layer.....	44
4.3 Testing the Model.....		46
CHAPTER FIVE.....		50
SUMMARY AND CONCLUSION.....		50
5.1 Summary.....		50
5.2 Conclusion.....		50
REFERENCES.....		52

ABSTRACT

Brain tumors are a serious health concern as they can have significant impacts on an individual's cognitive and physical abilities. Early detection of brain tumors is essential for successful treatment and improving patient outcomes. In this project, we propose the use of convolutional neural networks (CNNs) for brain tumor detection. CNNs are a type of deep learning algorithm that are particularly well suited for image classification tasks. By training a CNN on a large dataset of brain MRIs, we aim to develop a model that can accurately identify the presence of a brain tumor in a given MRI scan. Overall, our goal is to demonstrate the potential of CNNs for early and accurate detection of brain tumors, which has the potential to significantly improve patient outcomes and save lives.

CHAPTER ONE

INTRODUCTION

1.1 Background Information

Brain Tumor segmentation is one of the most crucial and arduous tasks in the field of medical image processing, as a human-assisted manual classification can result in inaccurate prediction and diagnosis. Moreover, it becomes a tedious task when there is a large amount of data present to be processed manually. Brain tumors have diversified appearance and there is a similarity between tumor and normal tissues and thus the extraction of tumor regions from images becomes complicated. According to data on the mortality rate of brain tumors, this cancer type is among the most worrisome and dangerous to humans. More than 1,000,000 persons are diagnosed with brain tumors each year worldwide, according to the International Agency for Research on Cancer (IARC), with has steadily increased the mortality rate. it is the second-most common cause of cancer deaths in infants and individuals under the age of 34. In recent times, doctors are following advanced methods to identify the tumor, which are more painful for the patients. To analyze the abnormalities in different parts of the body, CT (Computed Tomography) scan and MRI (Medical Reasoning Imaging) are two methods which are preferred by patients. Medical imaging techniques are used to view the inner portions of a human body for medical diagnosis. And medical image classification is one of the most challenging & affluent topics in the field of Image Processing. Medical image classification problems, tumor detection

or detection of Cancer is the most prominent one. In recent times we have seen a rise in interest in MRI-based medical image analysis for brain tumor investigations as a result of the rising demand for effective and impartial analysis of significant amounts of medical data.

Medical imaging refers to a number of techniques that can be used as non-invasive methods of looking inside the body. Medical image encompasses different image modalities and processes to image the human body for treatment and diagnostic purposes and hence plays a paramount and decisive role in taking actions for the betterment of the health of the people.

Image segmentation is a crucial and essential step in image processing which determines the success of a higher level of image processing. The primary goal of image segmentation in medical image processing is mainly tumor or lesion detection, efficient machine vision and attaining satisfactory result for further diagnosis. Improving the sensitivity and specificity of tumor or lesion has become a core problem in medical images with the help of Computer Aided Diagnostic (CAD) systems. It takes advanced computerized quantification and visualization techniques to analyze this wide range of image formats. Therefore, by eliminating the necessity for manually processing such a large amount of data, automatic brain tumor diagnosis from MRI pictures will be vital in this scenario.

A major problem in classifying MRI images by specific neural networks lies in the number of images in the database. In addition, MRI images are available on a variety of plane, so the option of using all the planes will expand the size of the database. Pre-processing is required before feeding the images to the neural network, one of the well-known advantage of use of convolutional neural networks (CNN) is that manual pre-processing and feature extraction is not required. The purpose of this study was to classify a given image into a healthy or malignant

through CNN. In this paper, we present a CNN model for brain image processing to healthy or malignant.

1.2 Motivation

Considering recent statistics on the mortality rate caused by brain tumors, i have selected brain tumor Detection and classification belonging to the field of medical image analysis. tumor Detection in medical images takes time because they depend on human judgment. The Experts in this field, such as radiologists, specialist doctors, examine CT scans, MRIs, PET scans and give decisions upon which the treatment depends. This whole process is long and tedious. Automated analysis of medical images can help reduce time and effort taken here and the workload of a human as performed by artificial neural networks.

1.3 Problem Definition

Brain tumors are commonly misdiagnosed because a physician failed to order further testing based on symptoms. Since the symptoms of brain tumors often mimic symptoms of other more common diseases, physicians often diagnose and prescribe treatment for another ailment. This can be dangerous for the patient, as the brain tumors are being allowed to grow and possibly affect further brain functions. In some cases, the patient dies before a proper diagnosis is made. Brain tumors can be malignant (cancerous), or benign (noncancerous). Even if the tumor is benign, the patient must usually undergo treatment, as any growth within the brain tissue can affect motor skills or other functions.

1.4 Goal and Objectives

The main objective of this project is to build a train that can predict whether the medical images contain a tumor or not. Primarily, dataset collection is the most important part to work on because brain tumor datasets are scarce and a bit complicated to acquire. Most of the research focused on definitive work like filtering and segmentation. Here we tried to establish a model which can accomplish all the fundamental and major necessary tasks to find a tumor and its properties. We proposed an efficient and effective method which helps in the segmentation and detection of the brain tumor without any human assistance, based on Convolutional Neural Network.

1.5 Scope of Research

This research focuses on the creation and training of an artificial neural network whose sole purpose is to detect and classify brain tumors and using the Convolution neural network model we are capable of getting a 93% correct diagnosis rate.

1.4 Research Methodology

The main stages involved in this project are Data Collection, Pre-processing and detection via neural network. Generally, accurate semi-automatic and automatic methods are required for detection. For these reasons, fully automatic segmentation system using CNN is used.

1.5 Research Significance

Due to the challenges motivating this work, the following significance will be realized:

- this neural network will be able to instantly detect tumor growth in the brain
- provide an indept reason for using CNN over other deep learning algorithms

CHAPTER TWO

LITERATURE REVIEW

This chapter gives an overview on brain tumors, why they are so deadly, and the problem with misdiagnosis. Then, it provides reviews on systems already in existence for automatically detecting brain tumors. Finally, it discusses related works and gives a brief summary of this chapter.

2.1 Brain tumors: An Overview

Ilhan (2019) states that a brain tumor occurs when abnormal cells form within the brain. Many different types of brain tumors exist. Some brain tumors are noncancerous (benign), while some brain tumors are cancerous (malignant) and some are pre-malignant. Cancerous tumors can be divided into primary tumors that start within the brain, and secondary tumors that have spread from somewhere else, known as brain metastasis tumors.

2.1.1 Classification Brain tumors

There are two types of brain tumors. The first is the Benign Tumor characterized as non-cancerous and the other one is Malignant Tumor- also known as Cancerous Tumor.

A. Benign tumors

Benign brain tumors are usually defined as a group of similar cells that do not follow normal cell division and growth, thus developing into a mass of cells that microscopically donot

have the characteristic appearance of a cancer (Silverstein and Alvin (2006)). These are the properties of a benign tumor:

- ❖ Most benign tumors are found by CT or MRI brain scans.
- ❖ Grows slowly, do not invade surrounding tissues or spread to other organs, and often have a border or edge that can be seen on CT scans.
- ❖ It can be life threatening because they can compress brain tissues and other structures inside the skull, so the term 'benign' can be misleading.

B. Malignant Tumors:

Malignant brain tumors contain cancer cells and often do not have clear borders. They are considered to be life threatening because they grow rapidly and invade surrounding brain Tissues(Strayer and Raphael, 2008) . These are the properties of a malignant tumor:

- Fast growing cancer that spreads to other areas of the brain and spine.
- A malignant brain tumor is either graded 3 or 4, whereas grade 1 or 2 tumors are usually classified as benign or non-cancerous.
- Generally these are more serious and often more fatal threat to life.

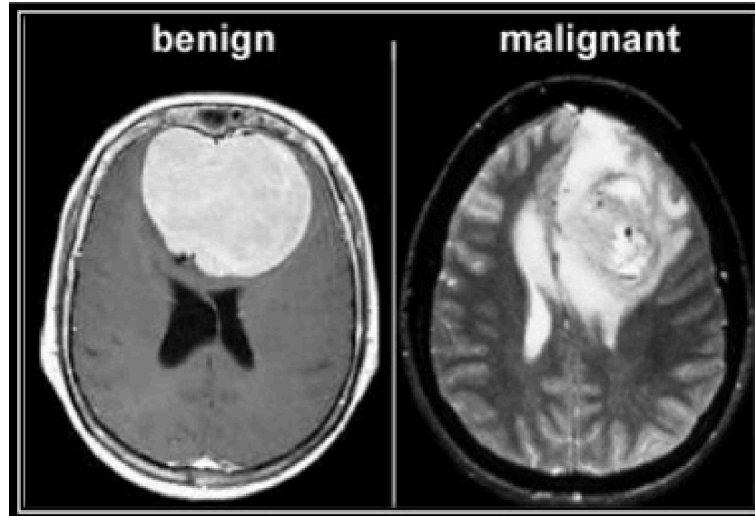


FIGURE 1.1 Benign Tumor (left) and Malignant Tumor (Right)

2.2 Related Work

In order to understand the research on tumor detection has gone, the following papers where heavily reviewed

Thillaikkarasi R et al. (2019), have worked on in-depth learning algorithm (kernel-based CNN) with M-SVM introduction to identify tumor automatically and efficiently. Image segmentation is done using M-SVM (Multi-class support vector machine) according with selected features. From an MRI scan, the tumor is isolated with the help of a kernel-based CNN-based method. The experimental results of the proposed method indicate that the method used may enable the brain tumor separation to reach approximately 84% accuracy in comparison with existing algorithms.

Todoroki et al. (2019), have proposed a system for the detection of liver transplants with different sizes in multiphase CT images using CNN's multi-channel protocol essential for transplantation. Test results reveal that memory points and dice coefficient can be significantly

improved, by reducing the number of false positives and the proposed method are compared with existing methods.

Zhang et al.(2020), have developed a methodology for detecting brain metastasis through MRI images. A total of 361 patients from 121 patients are used to train and test with fast region-based network with a convolutional neural network (Faster R-CNN). 270 scans of 73 patients were used for training; 488 planes are used in 91 scans of 48 test patients. Examination of data captured in MRI brain showed 96% sensitivity and 20 false metastases by scans. The results showed 87.1% sensitivity and 0.24 false metastases per unit.

Latif G et al.(2019), have proposed a method using 65-cases with 40,300 MR images obtained from the BRATS-2015 database. The work includes 26 Low-Grade Glioma (LGG) cases and 39 High-Grade Glioma (HGG) tumors. The proposed CNN model exceeds existing methods by obtaining an average accuracy of 98.77%.

Wang H et al.(2019), have worked on Focal cortical dysplasia (FCD) related to cortical developmental disorders. FCD automation detection strategies are proposed using MR images. All cortical patches are extracted from each axial plane, and these patches are divided into FCD and non-FCD using a deep convolutional neural network (CNN) with five layers of precision. Program quality testing, nine out of ten FCD images were successfully detected, and 85% of non-FCD images were correctly identified.

Kutlu et al.(2019), have proposed a method to differentiate liver and brain tumors using convolutional neural network (CNN). Test results show that the proposed method had higher performance than the other classifiers, such as the proximity K (KNN) and the vector support

machine (SVM). Results show that a 99.1% accuracy rate is obtained with a liver tumor classification and accuracy rate of 98.6% is achieved in the brain tumor phase.

Dina et al. (2014), introduced a model based on the Probabilistic Neural Network model related to Learning Vector Quantization. The model was evaluated on 64 MRI images, among which 18 MRI images were used as the test set, and the rest was used as a training set. The Gaussian filter smoothed the images. 79% of the processing time was reduced by the modified PNN method. A Probabilistic Neural Network based segmentation technique implemented by Othman et al. Principal Component Analysis (PCA) was used for feature extraction and also to reduce the large dimensionality of the data. The MRI images are converted into matrices, and then Probabilistic Neural Network is used for classification. Finally, performance analysis is done. The training dataset contained 20 subjects, and the test dataset included 15 subjects. Based on the spread value, accuracy ranged from 73% to 100%.

Rehman A et al.(2020), have proposed a method to detect brain tumors and their classification. The selected features are verified by the feed-forward neural network for final isolation. The three BraTS data sets for 2015, 2017, and 2018 are used for research, validation, and achieve accuracy of 98.32, 96.97, and 92.67%, respectively.

Devkota et al.(2009), established the whole segmentation process based on Mathematical Morphological Operations and spatial FCM algorithm which improves the computation time, but the proposed solution has not been tested up to the evaluation stage and outcomes as- Detects cancer with 92%, and classifier has an accuracy of 86.6%.

Yantao et al.(2019), resembled Histogram based segmentation technique. Regarding the brain tumor segmentation task as a three-class (tumor including necrosis and tumor, edema and normal tissue) classification problem regarding two modalities FLAIR and T1. The abnormal regions were detected by using a region-based active contour model on FLAIR modality. The edema and tumor tissues were distinguished in the abnormal regions based on the contrast enhancement T1 modality by the k-means method and accomplished a Dice coefficient and sensitivity of 73.6% and 90.3% respectively.

Sasikala et al.(2018),proposed a novel method for automatic cancer detection using DNN(deep neural network) for expertise detectionof glioblastomas. Its use final layer which implementationon fast segmentation, which is an order of 24 seconds to 3 minutes for entire region of lungs.

Seetha and Raja (2018), proposed the brain tumor classification is performed by using Fuzzy C-means (FCM) based segmentation and SVM and DNN based classification is carried out. This resulted in high computational period and low accuracy

Archa and Kumar. (2018), this research discussed the pre-processing consisting of intensity and batch normalization. It used bias field correction in CNN based method for the segmentation of brain tumors for MRI images. These images have a obstacle of intensity inhomogeneity (different intensity ranges for same sequences), which is avoided using NLTK. Median filters and Gaussian filers are also used. Clustering and segmentation process is

completed after patch pre-processing. Image enhancement using PCNN method is done along with Canny edge detection and edge detection using wavelet transform method.

Pathak et al.(2019), the primary objective of this research work is to construct a prototype for economical brain tumor classification and segmentation with high accuracy and low complexity. Marker based Watershed algorithm and Global thresholding are used for brain tumor segmentation. The training accuracy obtained is 98% along with a high validation accuracy and low validation loss.

From the literature survey; we can conclude that an ample amount of work is carried in classification of brain tumor images. But, only few researchers have reported their work using CNN. Hence, the proposed work is taken.

CHAPTER THREE

METHODOLOGY

3.1 Data set

A **data set** (or **dataset**) is a collection of data. In the case of tabular data, a data set corresponds to one or more database tables, where every column of a table represents a particular variable, and each row corresponds to a given record of the data set in question. The data set lists values for each of the variables, such as for example height and weight of an object, for each member of the data set. Data sets can also consist of a collection of documents or files.

3.1.1 Properties of data sets

Before performing any statistical analysis, it is essential to understand the nature of the data. We can use different Exploratory Data Analysis (EDA techniques), which helps to identify the properties of data, so that the appropriate statistical methods can be applied on the data. With the help of EDA techniques, we can check the following properties of the dataset.

- Centre of data
- Skewness of data
- Spread among the data members
- Presence of outliers
- Correlation among the data
- Type of probability distribution that the data follows

3.2 DEEP LEARNING

Deep Learning is a subset of machine learning algorithms that is very good at recognizing patterns but typically requires a large number of data. Deep learning excels in recognizing objects in images as it is implemented using three or more layers of artificial neural networks where each layer is responsible for extracting one or more features of the image .

3.2.1 Neuron

Artificial neurons, which resemble the neurons in the human brain, are the fundamental building block of neural networks. With weighted input signals and an activation function-based output signal, these artificial neurons are potent computing units. In a neural network, these neurons are dispersed among the various layers.

The three parameters of a neuron, namely:

Weight: When a signal (value) arrives, a neuron gets multiplied by a weight value. If a neuron has three inputs, it has three weight values which can be adjusted during training time.

Bias: It is an extra input to neurons and it is always 1, and has its own connection weight. This makes sure that even when all the inputs are none (all 0's) there is going to be an activation in the neuron.

Activation Function: Activation functions are also known as transfer functions. It helps in classification or partition. It uses a threshold, according to this threshold, we make the division into two or many classes.

3.2.2 Artificial Neural Network

Deep learning consists of artificial neural networks that are modeled on similar networks present in the human brain. These neural networks work in multiple layers so this kind of machine learning is called deep learning. Artificial neural networks are a way of calculating an output from an input (a classification) using weighted connections ("synapses") that are calculated from repeated iterations through training data. Each pass through the training data alters the weights such that the neural network produces the output with greater "accuracy" (lower error rate). The combination of working memory and speed is crucial when we're doing hundreds of thousands of matrix multiplications.

There are three layers of an artificial neural network and they are listed below:

Input Layer:

This layer consists of neurons and they just receive the inputs and pass it to the next layer. The number of layers in the input layer should be equal to the attributes or features in the dataset.

Hidden Layer:

In between input and output layer there are hidden layers based on the type of model. Hidden layers contain vast number of neurons. The neurons in the hidden layer apply transformations to the inputs before passing them to the next layer. As the network is trained the weights get updated, to be more predictive. The actual processing of the data is done via a system of weighted connections in the hidden layer. The hidden layers are linked to the output layer.

Output Layer:

The output layer is the predicted feature, or class in a classification problem, it basically depends in the type of the built model. The output layer gives the output based on the information passed from the hidden layer.

3.3 Types of Neural Networks

There are several kinds of artificial neural networks. These type of networks are implemented based on the mathematical operations and a set of parameters required to determine the output. Some of the popular neural networks are :

Feed-forward Neural Network: This neural network is one of the simplest forms of ANN, where the data or the input travels in one direction. The data passes through the input nodes and exit on the output nodes. This neural network may or may not have the hidden layers. In simple words, it has a front propagated wave and no back propagation by using a classifying activation function usually.

Radial Basis Function Neural Network: Radial basis functions consider the distance of a point with respect to the center. RBF functions have two layers, first where the features are combined with the Radial Basis Function in the inner layer and then the output of these features are taken into consideration while computing the same output in the next time-step which is basically a memory.

Kohonen Self Organizing Neural Network: A self-organizing map (SOM) is a type of artificial neural network (ANN) that is trained using unsupervised learning to produce

a low-dimensional (typically two-dimensional), discretized representation of the input space of the training samples, called a map, and is therefore a method to do dimensionality reduction.

Recurrent Neural Network: The Recurrent Neural Network works on the principle of saving the output of a layer and feeding this back to the input to help in predicting the outcome of the layer. This is also known as Long Short Term Memory.

Convolutional Neural Network: Convolutional neural networks are similar to feed forward neural networks, where the neurons have learn-able weights and biases.

Modular Neural Network: Modular Neural Networks have a collection of different networks working independently and contributing towards the output. Each neural network has a set of inputs which are unique compared to other networks constructing and performing sub-tasks. These networks do not interact or signal each other in accomplishing the tasks.

3.4 Deep learning models

There are various deep learning models which can be used for classification and they are discussed below;

3.4.1 Multi-layer Perceptron

A multi layer perceptron (MLP) is a feed forward artificial neural network that generates a set of outputs from a set of inputs. A MLP consists of at least three layers of nodes: an input layer, a hidden layer and an output layer [https://medium.com/@AI_with_Kain/understanding-of-multilayer-perceptron-mlp-8f179c4a135f]. Except for the input nodes, each node is a neuron that uses a nonlinear activation function. An MLP is characterized by several layers of input nodes connected as a directed graph between the input and output layers. MLP uses back-propagation for training the network. Multi-layer are most of the neural networks except deep learning. It uses one or two hidden layers.

3.4.2 Naive Bayes

Naive Bayes is a machine learning algorithm for classification problems. It is based on Bayes' probability theorem to predict the class of unknown data set. It predicts membership probabilities for each class such as the probability that given record or data point belongs to a particular class. The class with the highest probability is considered as the most likely class.

In a machine learning classification problem, there are multiple features and classes, say, $C_1, C_2, \dots, C_k$. The main aim in the Naive Bayes algorithm is to calculate the conditional probability of an object with a feature vector $x_1, x_2, \dots, x_n$ belongs to a particular class C_i . And the equation can be written as

$$P(C_i|x_1, x_2, \dots, x_n) = \frac{P(x_1, x_2, \dots, x_n|C_i) * P(C_i)}{P(x_1, x_2, \dots, x_n)}, \text{ for } 1 \leq i < k$$

or we can write,

$$P(C_i|x_1, x_2, \dots, x_n) = \left(\prod_{j=1}^{j=n} P(x_j|C_i) \right) * \frac{P(C_i)}{P(x_1, x_2, \dots, x_n)}, \text{ for } 1 \leq j \leq n$$

FIGURE3.1 (Native bayes Equation)

3.4.3 Support Vector Machine (SVM)

Support vector machines are supervised learning models with associated learning algorithms that analyze data used for classification and regression analysis.

Given a set of training examples, each marked as belonging to one or the other of two categories, an SVM training algorithm builds a model that assigns new examples to one category or the other, making it a non-probabilistic binary linear classifier [Ho TK (1998)]. An SVM model is a representation of the examples as points in space, mapped so that the examples of the separate categories are divided by a clear gap that is as wide as possible. New examples are then mapped into that same space and predicted to belong to a category based on which side of the gap they fall. In addition to performing linear classification, SVM can efficiently perform a non-linear classification using what is called the kernel trick, implicitly mapping their inputs into high dimensional feature spaces.

3.4.4 Random Forest

Random forest is a type of supervised machine learning algorithm based on ensemble learning. Ensemble learning is a type of learning where we join different types of algorithms or same algorithm multiple times to form a more powerful prediction model. The random forest algorithm combines multiple algorithm of the same type i.e. multiple decision trees. Random decision forests correct for decision trees' habit of over-fitting to their training set [Hastie, Trevor; Tibshirani (2008).]. It operates by constructing a multitude of decision trees at training time and outputting the class that is the mode of the classes (classification) or mean prediction (regression) of the individual trees

3.4.5 Convolutional Neural Network (CNN)

The basic idea about the Convolutional Neural Network was introduced by Kunihiro Fukushima in the 1980s. Convolutional Neural Networks (CNNs) are a category of Neural Networks that have proven very effective in areas such as image recognition and classification. Computer vision techniques are dominated by convolutional neural networks because of their accuracy in image classification. CNN is a class of deep, feed-forward artificial neural networks (where connections between nodes do not form a cycle) & use a variation of multi-layer perceptrons designed to require minimal pre-processing. CNN architectures make the explicit assumption that the inputs are images, which allows us to encode certain properties into the architecture. These then make the forward function more efficient to implement and vastly reduce the amount of parameters in the network. CNNs are made up of neurons that have learnable weights and biases. Each neuron receives some inputs, performs a dot product and optionally follows it with a non-linearity. The whole network still expresses a single

differentiable score function: from the raw image pixels on one end to class scores at the other. And they have a loss function on the last layer.

(A) Why CNN is different than simple Neural Network

Convolutional Neural Networks have a different architecture than regular Neural Networks. Regular Neural Networks transform an input by putting it through a series of hidden layers. Every layer is made up of a set of neurons, where each layer is fully connected to all neurons in the layer before and where neurons in a single layer function completely independently and do not share any connections between themselves. Finally, there is a last fully-connected layer, which is the output layer that represents the predictions. Regular Neural Networks do not scale well to full images. Convolutional Neural Networks are a bit different. First of all, the layers are organized in three dimensions: width, height and depth. Further, the neurons in one layer do not connect to all the neurons in the next layer but only to a small region of it. Lastly, the final output will be reduced to a single vector of probability scores, organized along the depth dimension.

Moreover, CNNs perform convolution operation in case of matrix multiplication.

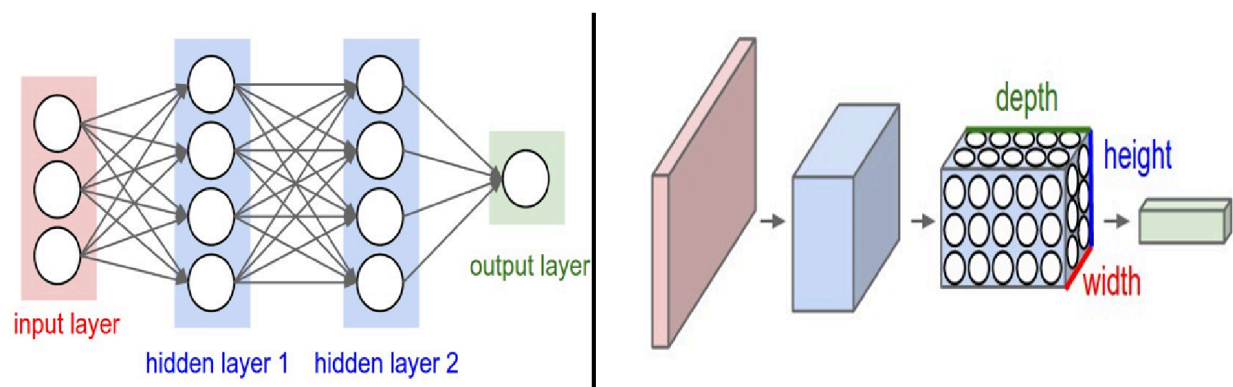


FIGURE 3.2(A basic Convolutional neural network)

In the diagram above, the left side represents a regular three Layer neural network. On the other hand, the right side of the diagram represents a CNN which arranges its neurons in three dimensions (width, height, depth). Every layer of a CNN transforms the 3D input volume to a 3D output volume of neuron activation's. In this example, the red input layer holds the image, so its width and height would be the dimensions of the image, and the depth would be three (Red, Green, Blue channels).

(I) The Convolution Operation

Convolutional Neural Networks perform a mathematical operation, known as convolution operation. Convolution is a mathematical operation on two functions (f and g) and it produces a third function. The convolution operation of f and g is denoted as $f * g$. It is defined as the integral of the product of the two functions after one is reversed and shifted. This operation is a particular kind of integral transform one is reversed and shifted. This operation is a particular kind of integral transform.

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau) d\tau$$

FIG 3.3 (Convolution operation)

There are three elements that enter into the convolution operation:

Input image: It is the image that is given as an input.

Feature detector: The feature detector is often referred to as a “kernel” or a “filter”.

Sometimes a 5*5 or a 7*7 matrix is used as a feature detector.

Feature map: The feature map is also known as an activation map. It is called feature map because it is also a mapping of where a certain kind of feature is found in the image.

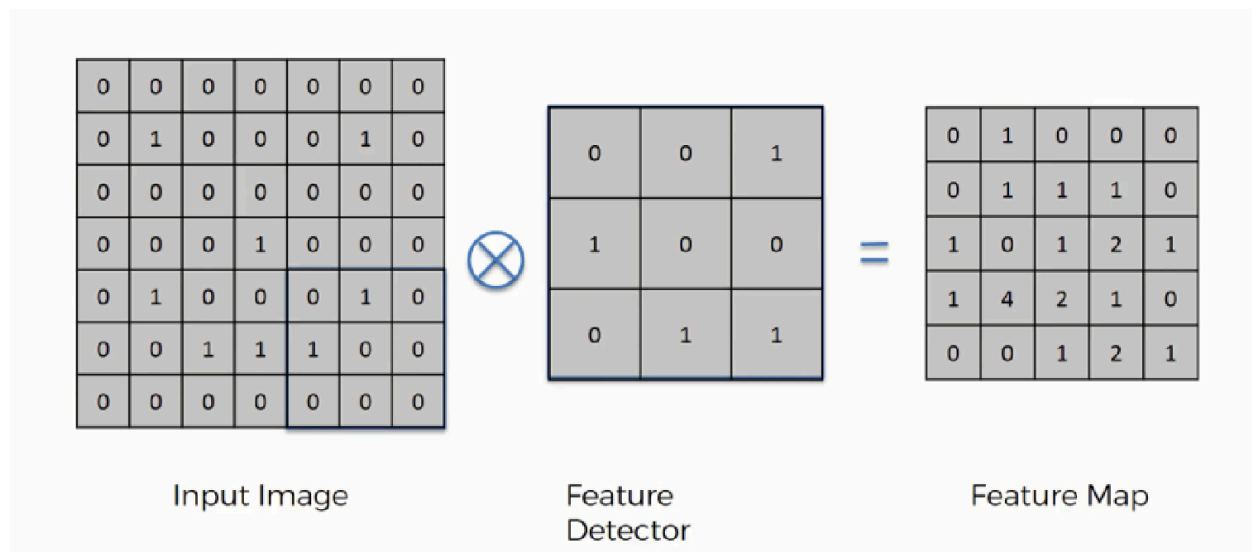


FIGURE 3.4 (convolution operation)

(i) The Convolution Arithmetic

Here the facts are shown how properties of an output image changes from input by some factors

Let, Input Image Size = I , (means width I and Height I i.e. image size is $I * I$)

Filter Size = F , (Filter is $F * F$)

Number of Filters = K

Number of Strides = S

Amount of Zero Padding = P

Then the output image size will be:

$$O = \frac{I - F + 2P}{S} + 1$$

(i) Forward Pass for Convolutional Layer

In forward pass of convolutional layer, a filter performs dot multiplication with all the parts of the input matrix one after another of filter size, then sum all the elements of the single dot product and adds the bias value with it and places the final value in the corresponding location of the output matrix for that dot product. This process continues across full input image with all filters and for each filter there comes an output.

(ii) Back-propagation for Convolutional Layer

In back-propagation, the cost function is first found out, then this cost function measures the displacement with the output. After that, applying gradient descent on this function will update the filter value of the previous layer. This process continues until it reaches to the input layer. The iteration of forward pass and back-propagation will continue until the network finds the desired output.

(B) Layer's Used to a Build CNN Model:

A simple CNN is a sequence of layers, and every layer of a CNN transforms one volume of activation's to another through a differentiable function. Three main types of layers used to build CNN architectures are:

Convolutional Layer:

The Convolutional layer is the core block of the Convolutional Neural Network. It has some special properties. It does most of the computational heavy lifting. The CNN layer's parameters consist of a set of learn-able filters. Every filter is small spatially (along width and height), but extends through the full depth of the input volume. For example, a typical 3X3 filter on a first layer of CNN might have size 5*5*3 (i.e. 5 pixels width and height, and 3 because images have depth 3, the color channels). During the forward pass, each filter is convolved across the width and height of the input volume and dot products are computed between the entries of the filter and the input at any position. As the filters are slid over the width and height of the input volume, a 2-dimensional activation map will be produced that gives the responses of that filter at every spatial position. Intuitively, the network will learn filters that activate when they see some type of visual feature such as an edge of some orientation. These activation maps are stacked along the depth dimension and the output volume is produced.

The Convolutional layer has some basic features , such as:

(i) Parameter Sharing:Parameter sharing is sharing of weights by all neurons in a particular feature map.

(ii)Local Connectivity (:Local connectivity is the concept of each neural connected only to a subset of the input image (unlike a neural network where all the neurons are fully connected)

These features help to reduce the number of parameters in the whole system and makes the computation more efficient.

Three hyper-parameters control the size of the output volume of the convolutional layer. These parameters are:

(1)Depth:The depth of input volume in first layer is the number of color channels of that input image. If the input image is a color image then the depth is 3 that is the red channel, the green channel and the blue channel. If the image is black and white or gray-scale, then the depth is 1. The depth of the output volume is the number of filters that we use in the input. The image below represents the depth which changes from 3 to 32 when we use 32 filters and the image size gets smaller.

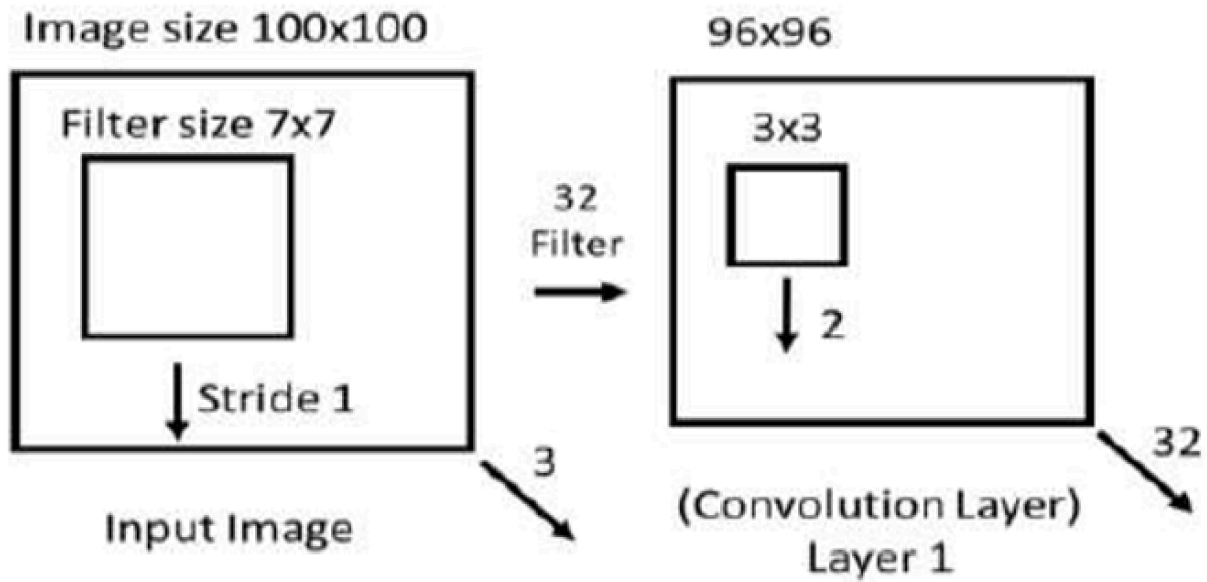


FIGURE 3.5 (Depth changing from 3 to 32 using 32 filters)

(2) Stride: Stride is used to slide along width and height of the input image. When the stride is 1, then we move the filters one pixel at a time. When the stride is 2, then the filters jump 2 pixels at a time as we slide them around.

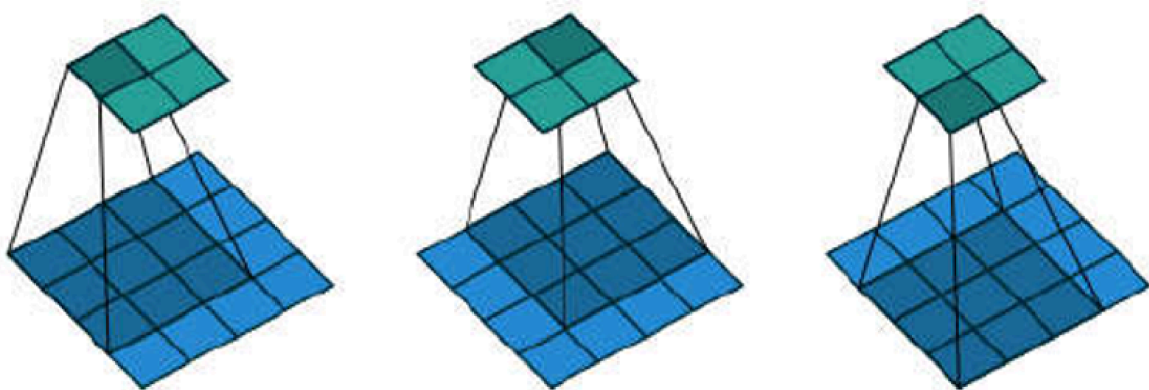


FIGURE 3.6 (Sliding filter along an input image)

(3) Zero Padding: Sometimes in the input layer, we pad the input image with zero that is called zero-padding. Zero padding allows us to control the size of the input layer. If we don't use zero-padding, sometimes some property from the edges can be lost.

0	0	0	0	0	0
0	35	19	25	6	0
0	13	22	16	53	0
0	4	3	7	10	0
0	9	8	1	3	0
0	0	0	0	0	0

FIGURE 3.7 (Zero-padding of an input)

(2) Fully-Connected Layer:

In fully connected layer, every neuron is connected to its previous layer neuron like the neural network. Its activation is also computed by matrix multiplication with its weight followed by bias as like neural network. Usually, fully connected layer is a column vector. A diagram of the fully connected layer is shown below

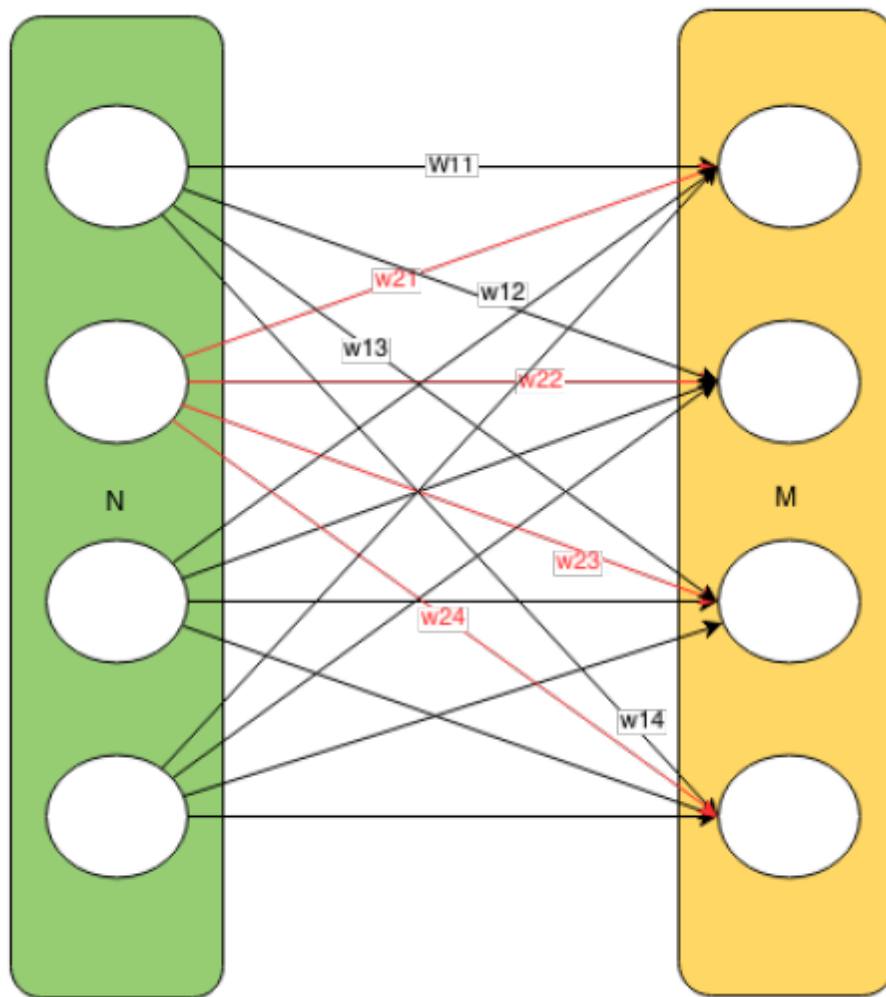


FIGURE 3.8 (Fully Connected Layer of CNN)

(3) Pooling Layer

Pooling layer is another building block of CNN. Usually pooling layer is placed after the convolutional layer. Its function is to progressively reduce the spatial size of the representation to

reduce the amount of parameters and computation in the network, and hence to also control over fitting. The Pooling Layer operates independently on every depth slice of the input and resizes it spatially. Pooling does not affect the depth dimension of the input volume. The task of pooling is done by summarizing the sub-regions of the input using some methods like taking the average or the maximum or the minimum value of the sub-regions only. These methods are called pooling functions.

Different Kinds of Pooling Functions: The pooling layer consists of some symmetric aggregation functions such as:

Max Pooling: It returns the maximum value from its rectangular neighborhood.

Average Pooling: It returns the maximum value from its rectangular neighborhood.

Weighted Average Pooling: It calculates its neighborhood weight based on distance from its center pixel.

L2 Norm Pooling: It returns the square root sum of its rectangular neighborhood.

In most of the CNN architectures, Max Pooling is used to reduce the computational cost.

Pooling Layer Arithmetic: Pooling layer works by sliding the window or filter across the input.

Let, Spatial Extent = f

Stride = s

window size = w

The equation shows that the output size from the pooling layer will be,

$$O = \left\lfloor \frac{w - f}{s} \right\rfloor + 1$$

- (i) **Activation Function:** Activation functions are used to introduce non-linearity to neural networks. It squashes the values in a smaller range. For example, a sigmoid activation function squashes values between a ranges 0 to 1.

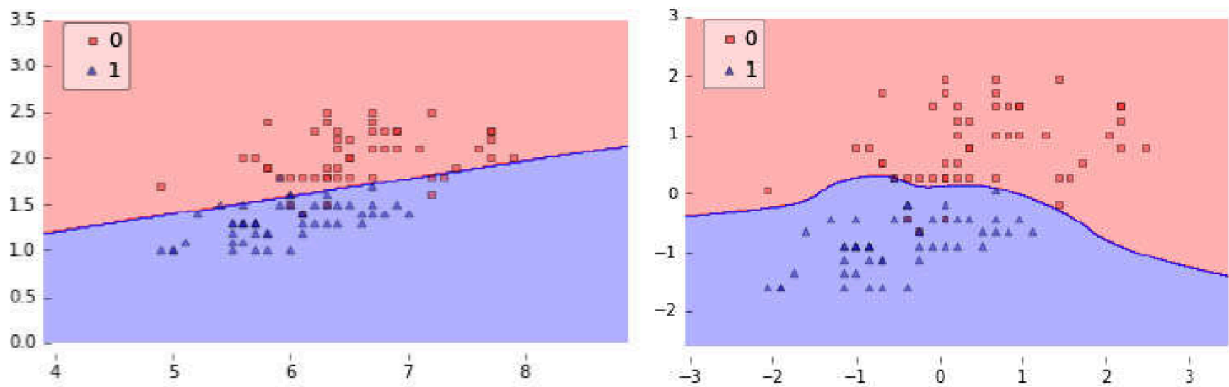


FIGURE 3.9 (Before and after applying activation function)

The figure above demonstrates the importance of activation function. The figure on the left side is the scenario of applying logistic regression without the application of activation function and the figure on the right side is the same scenario with the application of activation function.

- (ii) **Regularization Function:** In machine learning, regularization is a way to prevent overfitting. Regularization reduces over-fitting by adding a penalty to the loss function. By adding this penalty, the model is trained such that it does not learn interdependent set of features weights. If a simple model is not performing well at predicting due to poor

generalization and a complex model may not perform well due to over-fitting. In this case, regularization helps to choose the preferred complexity for the model. The two most important regularization techniques are Dropout and Batch Normalization.

Dropout: Dropout is an approach to regularization in neural networks which helps reducing interdependent learning amongst the neurons. Dropout forces a neural network to learn more robust features that are useful in conjunction with many different random subsets of the other neurons.

During the forward pass, Dropout temporarily removes some of the neurons. For example, if we fix dropout to 20%, then every 1 out of 5 neurons will be inactivated during the forward pass. During the backward pass, any weigh update will not be applied to these neurons.

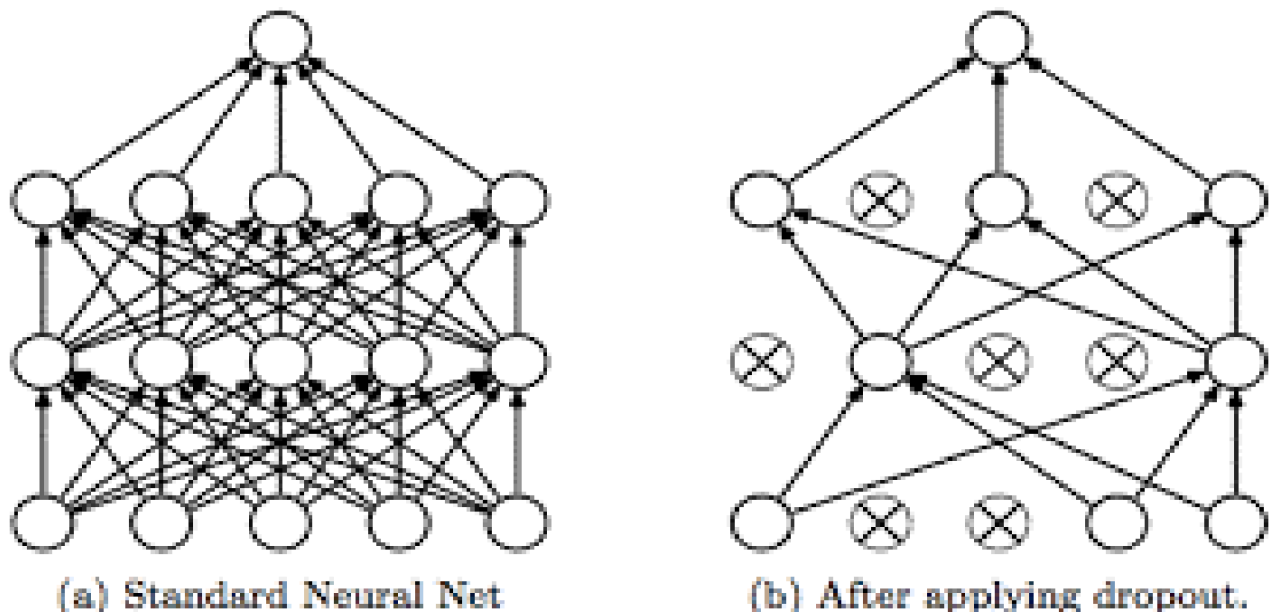


FIGURE 3.11 (Network before and after Dropout)

Batch Normalization: Batch Normalization is a method to reduce the internal covariate shift in neural networks, leading to the possible usage of higher learning rates. In principle, the method adds an additional step between the layers, in which the output of the layer before is normalized. BN further prevents smaller changes to the parameters to amplify and thereby allows higher learning rates, making the network even faster. Batch normalization (BN) consists of two algorithms. The first algorithm is the transformation of the original input of a layer x to the shifted and normalized value y . The second algorithm is the overall training of a batch-normalized network.

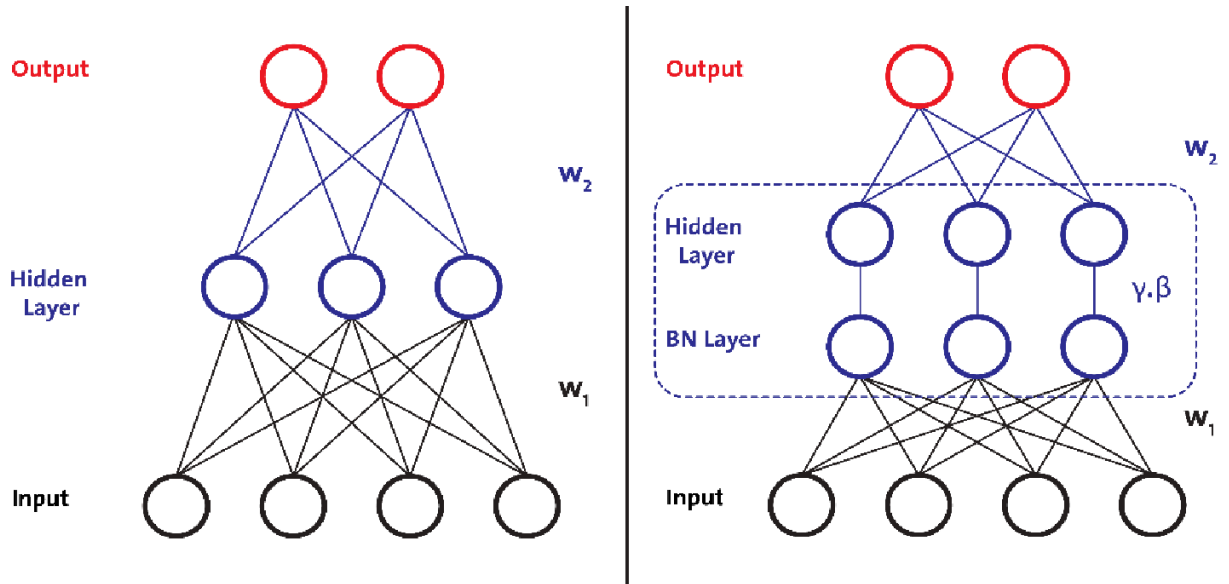


FIGURE 3.12 (A Neural Network before and after Batch Normalization)

- (iii) **Hyper-parameters of CNN architecture:** In our proposed model, we used different types of hyper-parameters. Following is a brief discussion about the hyper-parameters.

Bias: Bias is an error from erroneous assumptions in the learning algorithm. High bias can cause an algorithm to miss the relevant relations between features and target outputs (under-fitting).

glorot_uniform: It is named as Xavier normal initializer. It draws samples from a uniform distribution within $[-limit, limit]$.

Learning rate: The learning rate or step size in machine learning is a hyper-parameter which determines to what extent newly acquired information overrides old information.

Beta_1 and Beta_2: The algorithm calculates an exponential moving average of the gradient and the squared gradient, and the parameters β_1 and β_2 control the decay rates of these moving averages.

Epsilon: It is a very small number to prevent any division by zero in the implementation.

Dropout or Decay weight: A simple and powerful regularization technique for neural networks and deep learning models is dropout.

Amsgrad: AMSGrad (in Keras, it is controlled by setting `amsgrad = True` for Adam optimizer), which uses the maximum of past squared gradients in order to allow the rarely-occurring minibatches with large and informative gradients to have greater influence on the overall direction, otherwise diminished by exponential averaging in plain Adam.

Epoch: Epoch is defined as the number forward and backward pass of all training examples.

Batch Size: The number of training examples in one forward/backward pass.

(C) Limitations of CNN: CNNs only capture local ‘spatial’ patterns in data. If the data can’t be made to look like an image, CNNs are less useful. Moreover, CNNs are very bad at encoding different representations of pose and orientation within themselves. In CNNs, the orientations and their surroundings are not taken into account..

CHAPTER FOUR

SYSTEM DEVELOPMENT

System development is the process of defining, testing and implementing a new software application or program.

4.1 Software Implementation Tools

Software tools for implementation of the service solution are highlighted below:

- a. **Implementation Languages:** Python is a high-level, general-purpose programming language. Its design philosophy emphasizes code readability with the use of significant indentation. Python is dynamically-typed and garbage-collected. It supports multiple programming paradigms, including structured, object-oriented and functional programming.
- b. **Packages:** the model was trained using various packages and libraries listed below:
 - (i) **Tensor Flow:** TensorFlow is a free and open-source software library for machine learning and artificial intelligence. It can be used across a range of tasks but has a particular focus on training and inference of deep neural networks[Wikipedia].
 - (ii) **Keras:** Keras is an open-source software library that provides a Python interface for artificial neural networks. Keras acts as an interface for the TensorFlow library.

- (iii) **Open CV:**OpenCV is a library of programming functions mainly aimed at real-time computer vision. Originally developed by Intel, it was later supported by Willow Garage then Itseez. The library is cross-platform and free for use under the open-source Apache 2 License
- (iv) **OS:**The OS module in Python provides functions for interacting with the operating system. OS comes under Python's standard utility modules. This module provides a portable way of using operating system-dependent functionality. The `*os*` and `*os.path*` modules include many functions to interact with the file system[Geeks for Geeks].
- (v) **PIL:**Python Imaging Library is a free and open-source additional library for the Python programming language that adds support for opening, manipulating, and saving many different image file formats.
- (vi) **Numpy:** NumPy is a library for the Python programming language, adding support for large, multi-dimensional arrays and matrices, along with a large collection of high-level mathematical functions to operate on these arrays.
- (vii) **Sklearn:**scikit-learn is a free software machine learning library for the Python programming language. It features various classification, regression and clustering algorithms


```

import cv2
import os
from PIL import Image
import tensorflow as tf
from tensorflow import keras
import numpy as np
from sklearn.model_selection import train_test_split
from keras.utils import normalize
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Activation, Dropout, Flatten, Dense

```

FIGURE 4.1 (snippet of code for packages used)

C. Operating System: The model was trained and tested using a service operating system- Windows 10 64-bit. . Windows 10 has great support for seamless machine to application interaction where everything runs as a service, making it a perfect choice for deploying the software.

4.2 Training the Model

Convolutional Neural Network is broadly used in the field of Medical image processing. Over the years lots of researchers tried to build a model which can detect the tumor more efficiently. It is a class of deep neural networks which is applied to interpreting visual imagery. A fully-connected neural network can detect the tumor, but because of parameter sharing and sparsity of connection, we adopted the Convolutional Neural Network (CNN) for our model. The work the work flow for the proposed model is Below

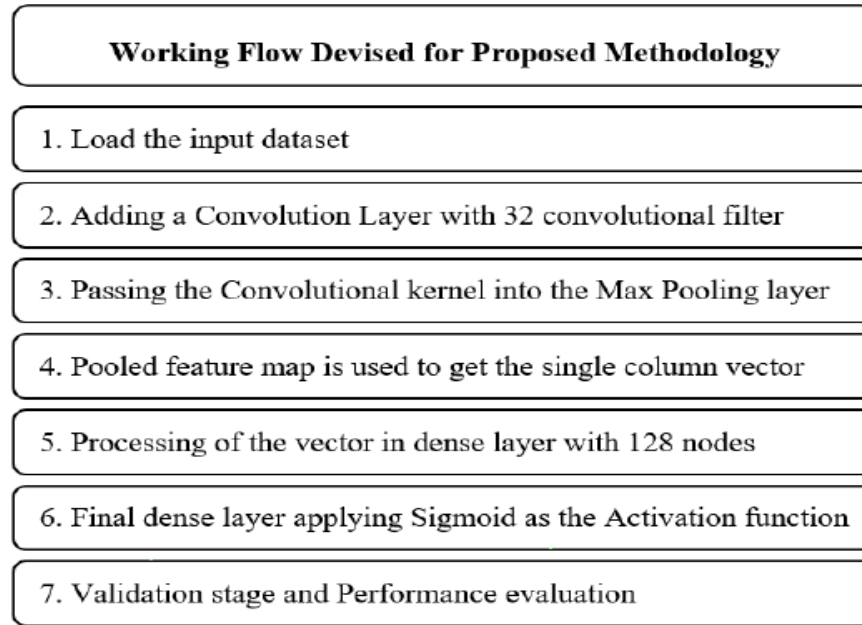


FIGURE4.2 (Work flow of proposed CNN model)

4.2.1 Acquiring the Dataset

We used the benchmark dataset in the field of Brain Tumor Segmentation, and that is BRATS dataset. BRATS is a Multimodal Brain Tumor Segmentation dataset named after MICCAI (The Medical Image Computing and Computer Assisted Intervention Society). The dataset is a labeled dataset differentiating tumor and non-tumor. The dataset breaks down into two segments no tumor and yes tumor. All the images are from three categories- T1- weighted, T1-weighted and FLAIR (Fluid Attenuated Inversion Recovery). Both sets consist of two classes. One is for tumorous MRI (class-1) and the other one is for Non-tumorous MRI (class-0). there are 1500 tumor MRI images and 1500 non-tumor MRI images, 30 predictions images were acquired from a private clinic. A final dataset was acquired from david Roberts a user account on the website kaggle for final testing consisting of 200 MRI images

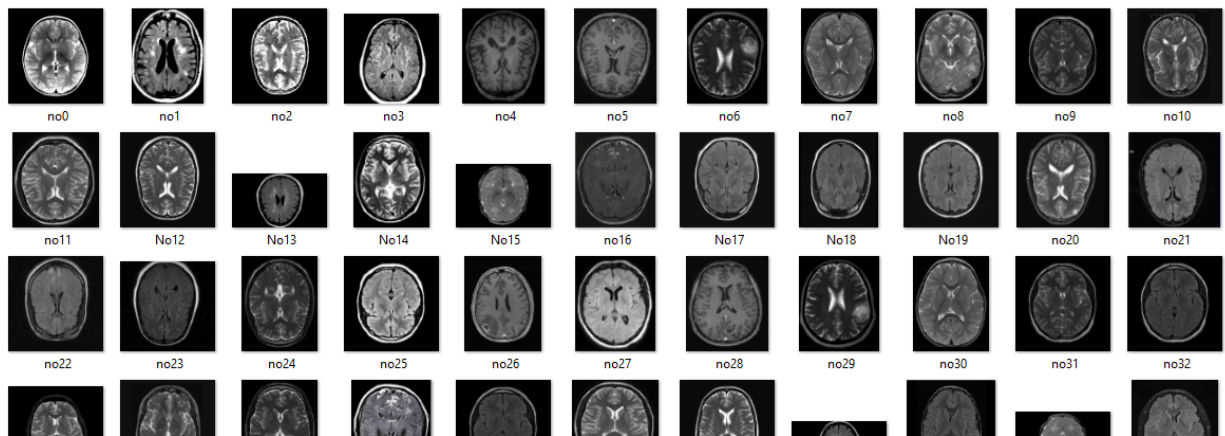


FIGURE 4.3 (No tumor MRI's)

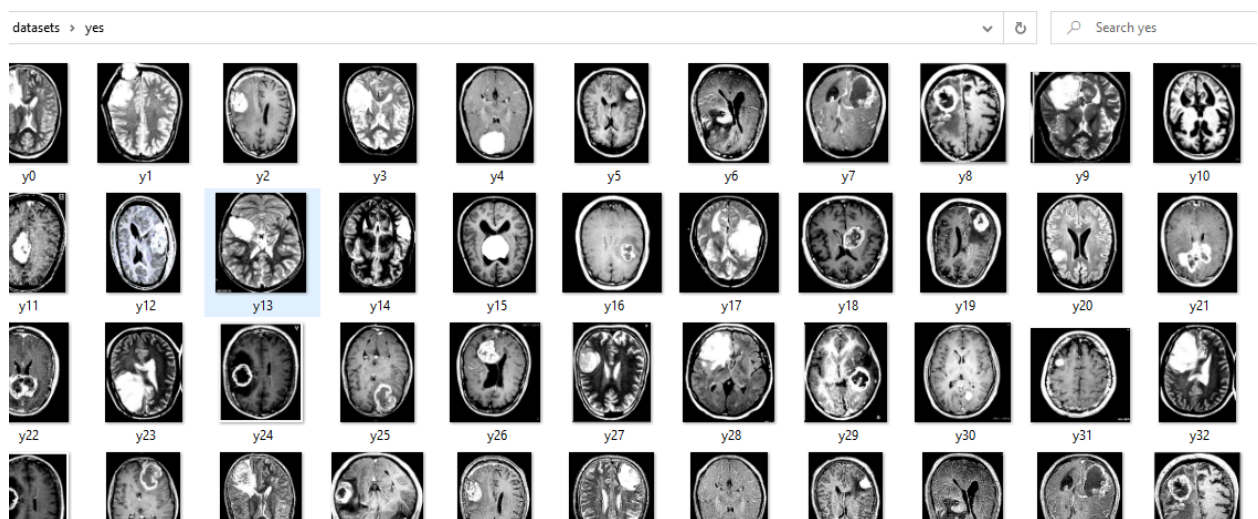


FIGURE 4.4 (Yes tumor MRI's)

```

11
12
13 image_directory='datasets/'
14
15
16 no_tumor_images=os.listdir(image_directory+'no/')
17 yes_tumor_images=os.listdir(image_directory+'yes/')
18 dataset=[]
19 label=[]
20 INPUT_Size= 64
21

```

FIGURE 4.5 (Snippet with dataset directory and input size)

4.2.2 Convolutional Layer

A Convolutional layer is the core building block of a CNN model. Using convolutional layer as the opening layer, an input shape of the MRI images is generated which is $64*64*3$, converting all the images into a same dimension. After accumulating all the images in the same aspect, we created a convolutional kernel that is convoluted with the input layer administering with 32 convolutional filters of size $3*3$ each with the support of 3 channels tensors. Rectified Linear Unit (ReLU) is used as an activation function. The input volume has size $64*64*3$ which means 64 pixels width, 64 pixels height, depth of 3 and the filter size is $3*3$. Then each neuron in the Convolutional Layer will have weights to a $3*3*3$ region in the input volume, for a total of $3*3*3 = 27$ weights and one will be added for bias parameter. There are three hyper parameters which we will evaluate and those are- depth, stride and zero-padding. For our model, the

input volume size is $64 \times 64 \times 3$, filter size is 3×3 so the spatial extent or filter size is 3. We did not use padding in the border so that Zero Padding, and stride is 1. Using stride as 1 grant us to do all the spatial down-sampling in the pooling layers, with the CONV layers only transforming the input volume depth-wise. Now using the convolutional layer equation we find the dimension of convolutional layer which is 62, 62 and 32 respectively. After Convolutional layer, we introduced the max pooling layer as the second layer.

```
--
INPUT_SIZE= 64

for i , image_name in enumerate(no_tumor_images):
    if(image_name.split('.')[1]=='jpg'):
        image=cv2.imread(image_directory+'no/'+ image_name)
        image=Image.fromarray(image, 'RGB')
        image=image.resize((INPUT_SIZE,INPUT_SIZE))
        dataset.append(np.array(image))
        label.append(0)

for i , image_name in enumerate(yes_tumor_images):
    if(image_name.split('.')[1]=='jpg'):
        image=cv2.imread(image_directory+'yes/'+ image_name)
        image=Image.fromarray(image, 'RGB')
        image=image.resize((INPUT_SIZE,INPUT_SIZE))
        dataset.append(np.array(image))
        label.append(1)

model=Sequential()

model.add(Conv2D(32, (3,3), input_shape=(INPUT_SIZE, INPUT_SIZE, 3)))
model.add(Activation('relu'))
```

FIGURE 4.6 (SNIPPET OF CONVOLUTIONAL LAYER)

4.2.3 Max Pooling Layer

The main focus of pooling layer is to progressively reduce the spatial size of the representation in order to reduce the number of parameters and computational task in the network. It can control over-fitting because it can scale down the parameters. Using the max pooling layer, it can operate independently on every depth slice of the input and resizes it spatially. Working on the Brain MRI image can also cost the contamination of the over-fitting and for this Max Pooling layer perfectly works for this perception. So, for our input image, we used MaxPooling2D for the proposed model. This convolutional layer runs on 31*31*32 dimension. Input size is 62*62*32 and after finding out the dimension of max pooling layer using the equation, we got the output volume size of 31*31*32. Because of dividing the input images in both spatial dimensions, the pool size is (2, 2) which means a tuple of two integers by which to downscale by vertically and horizontally.

```
model.add(MaxPooling2D(pool_size=(2,2)))
```

FIGURE 4.7 (snippet of the pooling layer/filter)

4.2.4 Flatten Layer

After the pooling layer, a pooled feature map is obtained. Flatten layer is one of the essential layers after the pooling because we have to transform the whole matrix representing the input images into a single column vector and its imperative for processing. It is then fed to the Neural Network for the processing. The dimension of this layer is $31*31*32 = 30752$.

4.2.4 Fully Connected Layer

Two fully connected layers were employed- Dense-1 and Dense-2 represented the dense layer. The dense function is applied in Keras for the processing of the Neural Network, and the obtained vector works as an input for this layer.

There are 128 nodes in the hidden layer. Because the number of dimension or nodes proportional with the computing resources we need to fit our model we kept it as moderate as possible and for this perspective 128 nodes gives the most substantial result. ReLU is used as the activation function because of showing better convergence performance.

After the first dense layer, the second fully connected layer was used as the final layer of the model. In this layer, we used the sigmoid function as activation function where the total number of the node is one because we need to lower the uses of computing resources so that a more significant amount assuages the execution time. There is a chance of hampering the learning in deep networks for using of the sigmoid as the activation function So we scale down the sigmoid function, and the number of the nodes is much lesser and easy to handle for this deep network.

```
model.add(Flatten())
model.add(Dense(64))
model.add(Activation('relu'))
model.add(Dropout(0.5))
model.add(Dense(1))
model.add(Activation('softmax'))

# Binary CrossEntropy= 1
# Categorical Cross Entropy = 2, soft max

model.compile(loss='crossentropy', optimizer='adam', metrics='accuracy')
```

```
model.fit(x_train, y_train,
          batch_size=16,
          verbose=1, epochs=10,
          validation_data=(x_test, y_test),
          shuffle=False)
```

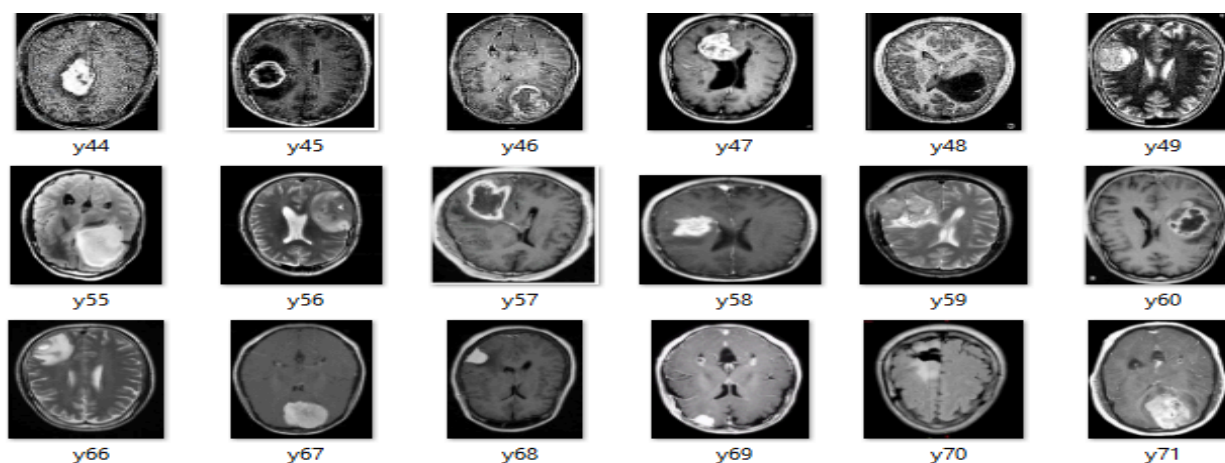
FIGURE 4.8(Snippet of fully connected layer)

```
94 model.fit(x_train, y_train,
PROBLEMS OUTPUT TERMINAL Python + - [ ] [X] ^ X
Epoch 6/10
150/150 [=====] - 10s 64ms/step - loss: 0.1140 - accuracy: 0.9675 -
  val_loss: 0.1142 - val_accuracy: 0.9650
Epoch 7/10
150/150 [=====] - 10s 64ms/step - loss: 0.0709 - accuracy: 0.9796 -
  val_loss: 0.1313 - val_accuracy: 0.9667
Epoch 8/10
150/150 [=====] - 10s 65ms/step - loss: 0.0479 - accuracy: 0.9846 -
  val_loss: 0.1157 - val_accuracy: 0.9683
Epoch 9/10
150/150 [=====] - 10s 64ms/step - loss: 0.0450 - accuracy: 0.9842 -
  val_loss: 0.1060 - val_accuracy: 0.9750
Epoch 10/10
```

FIGURE 4.9 (screenshot of model compilation)

4.3 Testing the Model

With the model now trained it was then tested to see how accurate its predictions are, to do this two datasets were used, the first was the initial dataset which was used to train the model with showed a 99% accuracy rating even though it was trained using 80% of the images from the data set .



```

2022-12-10 04:14:35.358734: I tensorflow/core/platform/cpu_feature_guard.cc:193]
This TensorFlow binary is optimized with oneAPI Deep Neural Network Library (on
eDNN) to use the following CPU instructions in performance-critical operations:
AVX
To enable them in other operations, rebuild TensorFlow with the appropriate compiler flags.
1/1 [=====] - 0s 169ms/step
[0]
PS C:\Users\ISAH JEFF\Desktop\datasets> & "c:/Users/ISAH JEFF/Desktop/datasets/.venv/Scripts/python.exe" "c:/Users/ISAH JEFF/Des
ktop/datasets/MainTest.py"
2022-12-16 04:14:58.822793: I tensorflow/core/platform/cpu_feature_guard.cc:193] This TensorFlow binary is optimized with oneAPI
Deep Neural Network Library (oneDNN) to use the following CPU instructions in performance-critical operations: AVX
To enable them in other operations, rebuild TensorFlow with the appropriate compiler flags.
1/1 [=====] - 0s 161ms/step
[1]

```

Activate Windows
Go to Settings to activate

FIGURE 4.9 (images with prediction results)

To then tests the models transparency a new data set was taken and ran through the model, though being a smaller size with 100 images combined, however when it was tested against the CNN model the results presented were different. Now the images showed a 99%

no_tumor prediction rate but a 93% yes tumor predictions.

```
1/1 [=====] - 4s 4s/step
[1]
PS C:\Users\ISAH JEFF\Desktop\datasets> & "c:/Users/ISAH JEFF/Desktop/datasets/.venv/Scripts/python.exe" "c:/Users/ISAH JEFF/Desktop/datasets/MainTest.py"
2022-12-18 16:43:55.094276: I tensorflow/core/platform/cpu_feature_guard.cc:193] This TensorFlow binary is optimized with oneAPI Deep Neural Network Library (oneDNN) to use the following CPU instructions in performance-critical operations: AVX
To enable them in other operations, rebuild TensorFlow with the appropriate compiler flags.
1/1 [=====] - 0s 172ms/step
[0]
PS C:\Users\ISAH JEFF\Desktop\datasets> & "c:/Users/ISAH JEFF/Desktop/datasets/.venv/Scripts/python.exe" "c:/Users/ISAH JEFF/Desktop/datasets/MainTest.py"
2022-12-18 16:44:28.090008: I tensorflow/core/platform/cpu_feature_guard.cc:193] This TensorFlow binary is optimized with oneAPI Deep Neural Network Library (oneDNN) to use the following CPU instructions in performance-critical operations: AVX
To enable them in other operations, rebuild TensorFlow with the appropriate compiler flags.
1/1 [=====] - 0s 158ms/step
[1]
```

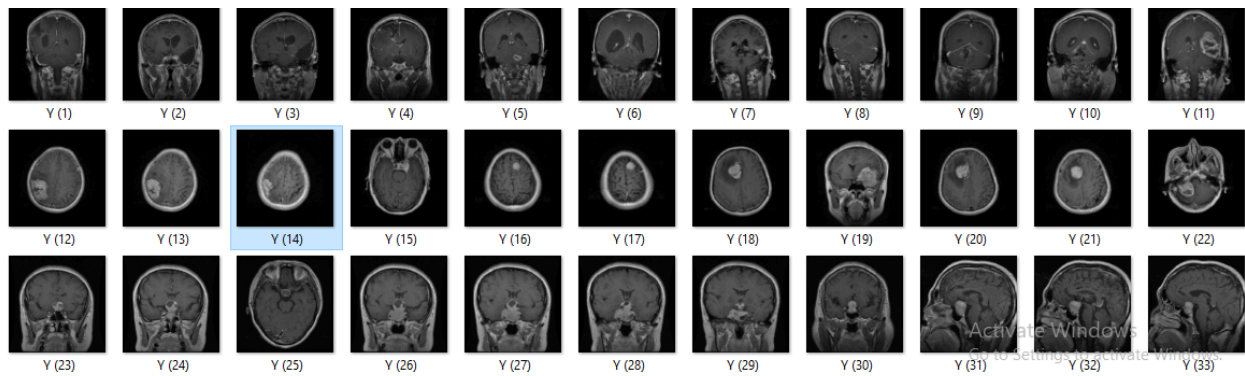


FIGURE 4.10 (tumor images of newly compiled MRI's)

```

MainTest.py > ...
1  import cv2
2  from keras.models import load_model
3  from PIL import Image
4  import numpy as np
5  import os
6  import glob
7
8
9
10 model=load_model('BrainTumorModel.h5')
11
12 image=cv2.imread('C:\\Users\\ISAH JEFF\\Desktop\\datasets\\datasets\\yes\\y42.jpg')
13
14 |
15 img=Image.fromarray(image)
16
17 img=img.resize((64,64))
18
19 img=np.array(img)
20
21 input_img=np.expand_dims(img, axis=0)
22
23
24 result=np.argmax(model.predict(input_img), axis=-1)
25
26 print(result)
27
28

```

FIGURE 4.11 (entire code for testing the model)

Similar results were gotten when a similar model was proposed by (Jayashree Shedbalkar in 2021).

Of course applying the model in the field would take a bit more work as the accuracy of the predictions are a function of the data set used to train the model, applying this model in the field would require a graphic user interface to make it useable even without programmers being involved. the result of the model were compared against similar models which were also used to detect and classify brain tumors.

Parmar (2012) designed his own model using the CNN algorithm but due to the size of the data set he used which was procured from an online data collection website (ITSY) he was

able to get an accuracy rating of 80% with very similar configurations used in the models training,

Seetha and Raja (2018), proposed the brain tumor classification is performed by using Fuzzy C-means (FCM) based segmentation and SVM and DNN based classification is carried out. This in turn resulted in longer training period and accuracy rating of as low as 48%, whereas the model proposed in this research produces over a 99% accuracy rating.

Thillaikkarasi R etal. (2019), worked on in-depth learning algorithm (kernel-based CNN) with M-SVM introduction to identify tumor automatically and efficiently. Which turned out a 97% accuracy rating .

CHAPTER FIVE

SUMMARY AND CONCLUSION

5.1 Summary

Decisions made concerning our health is of utmost importance and in the past these decisions have not been made in the most efficient way and these decisions would be best handled without unnecessary errors which can occur due to fatigue, emotional bias, laziness And more which can all happen when handled by professional doctors. with a loss life due to erroneous prediction being at an all time high it would be best to remove this decision from their hands and create new innovative ways to correctly make these choices. One of such innovations

is the brain tumor detection model which has been proposed in this project, with a lot of tumor deaths been directly linked to wrong diagnosis made by professionals. Human error can be eliminated by implementing such models into the field. as our model boasts of almost a 98% correct prediction rate it would definitely help to reduce the number of wrong diagnosis made with regards of brain tumor classification

5.2 Conclusion

The project examined new method based on the Convolutional Neural Network for tumor detection from brain images is acquired from various data sets. The CNN model has been tested and is capable of detecting a tumor. The CNN is very useful for selecting an auto-feature in medical images as was seen when it was trained with over 3000 MRI images and tested not only against them but when tested against other images gotten not used to train it were, it still produced positive results. It is valid to conclude that introducing a brain tumor detection model built using the CNN will immensely help in providing an efficient and worthy basis for making correct predictions on brain tumor classification

REFERENCES

“General Information About Adult Brain Tumors”. NCI. Last Accessed Date: May 10, 2019.

Nunn, Laura Silverstein; Silverstein, Alvin; Silverstein, Virginia B. (2006). “Cancer. Brookfield, Conn: Twenty-First Century Books”. pp. 11-12. ISBN 0-7613-2833-5.

Khurram Shehzad, Imran Siddique, Obed Ullah Memon, “Efficient Brain Tumor Detection Using Image Processing Techniques”, International Journal of Scientific & Engineering Research, December 2016.

Prof. Vrushali Borase., Prof. Gayatri Naik. and Prof. Vaishali Londhe, “Brain MR Image Segmentation for Tumor Detection using Artificial Neural”, International Journal Of Engineering And Computer Science, 2017.

Umit Ilhan, Ahmet Ilhan, “Brain tumor segmentation based on a new threshold approach,” Procedia Computer Science, ISSN: 1877-0509, Vol: 120, Page: 580-587,, 2017

Miss. Shrutika Santosh Hunnur, Prof. Akshata Raut, Prof. Swati Kulkarni, “Implementation of Image Processing for Detection of Brain Tumors”, IEEE, 2017.

Hussain Z, Gimenez F, Yi D, Rubin D. Differential Data Augmentation Techniques for Medical Imaging Classification Tasks. AMIA Annu Symp Proc. 2018

GeeksforGeeks. (n.d.). Convolutional Neural Network (CNN) in Machine Learning. Retrieved [insert date accessed, 11/12/2022], from [https://www.geeksforgeeks.org/convolutional-neural-network-cnn-in-%20machine-learning/]

APPENDIX

MODEL.TRAIN

```
import cv2
import os
from PIL import Image
import tensorflow as tf
from tensorflow import keras
import numpy as np
from sklearn.model_selection import train_test_split
from keras.utils import normalize
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Activation, Dropout, Flatten, Dense

image_directory='datasets/'

no_tumor_images=os.listdir(image_directory+'no/')
yes_tumor_images=os.listdir(image_directory+'yes/')
dataset=[]
label=[]
INPUT_SIZE= 64

for i , image_name in enumerate(no_tumor_images):
    if(image_name.split('.')[1]=='jpg'):
        image=cv2.imread(image_directory+'no/'+ image_name)
        image=Image.fromarray(image, 'RGB')
        image=image.resize((INPUT_SIZE,INPUT_SIZE))
        dataset.append(np.array(image))
```

```

label.append(0)

for i , image_name in enumerate(yes_tumor_images):
    if(image_name.split('.')[1]=='jpg'):
        image=cv2.imread(image_directory+'yes/'+ image_name)
        image=Image.fromarray(image, 'RGB')
        image=image.resize((INPUT_SIZE,INPUT_SIZE))
        dataset.append(np.array(image))
        label.append(1)

dataset=np.array(dataset)
label=np.array(label)

#Reshape = (n, image_width, image_height, n_channels)

x_train, x_test, y_train, y_test=train_test_split(dataset, label,
test_size=0.2,random_state=0)

#print(x_train.shape)
#print(y_train.shape)

#print(x_test.shape)
#print(y_test.shape)

x_train=normalize(x_train, axis=1)
x_test=normalize(x_test, axis=1)

# Model construction

```



```

model=Sequential()

model.add(Conv2D(32, (3,3), input_shape=(INPUT_SIZE, INPUT_SIZE, 3)))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2,2)))

model.add(Conv2D(32, (3,3), kernel_initializer='he_uniform'))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2,2)))

model.add(Conv2D(64, (3,3), kernel_initializer='he_uniform'))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2,2)))

model.add(Flatten())
model.add(Dense(64))
model.add(Activation('relu'))
model.add(Dropout(0.5))
model.add(Dense(1))
model.add(Activation('softmax'))

# Binary CrossEntropy= 1
# Categorical Cross Entropy = 2, soft max

model.compile(loss='crossentropy', optimizer='adam', metrics='accuracy')

model.fit(x_train, y_train,
batch_size=16,
verbose=1, epochs=10,

```

```
validation_data=(x_test, y_test),  
shuffle=False)
```

```
model.save('BrainTumorModel.h5')
```

MODEL TEST

```

import cv2
from keras.models import load_model
from PIL import Image
import numpy as np
import os
import glob

model=load_model('BrainTumorModel.h5')

image=cv2.imread('C:\\Users\\ISAH JEFF\\Desktop\\datasets\\datasets\\yes')
image=cv2.imread('C:\\Users\\ISAH JEFF\\Desktop\\datasets\\datasets\\no')

img=Image.fromarray(image)

img=img.resize((64,64))

img=np.array(img)

input_img=np.expand_dims(img, axis=0)

result=np.argmax(model.predict(input_img), axis=-1)

print(result)

```